

TECNOLOGIAS INALAMBRICAS

Proyecto Final

Centro Arcade con ESP32, NFC/RFID, WiFi y SQL Server en Docker

Integrantes:

- Eduardo Cadengo Lopez
- Itzel Citlalli Martell De La Cruz
- Damian Alexander Diaz Pina

05 de junio del 2026

Índice

1. Introducción	2
2. Marco Teórico e Investigación	3
2.1 NFC y RFID	3
2.2 ESP32 y WiFi	3
2.3 Modulo PN532.....	3
2.4 Docker y SQL Server.....	3
2.5 Node.js como servidor intermediario.....	4
2.6 Python y tkinter para el videojuego	4
3. Arquitectura del Sistema	4
4. Desarrollo del Proyecto	5
4.1 Configuración del servidor en la Mac.....	5
4.2 Configuración del Firewall en Mac.....	5
4.3 Código del ESP32	6
Configuración de pines y librerías	6
Lectura de tarjeta y envío de UID.....	7
Recepción del puntaje y envío al servidor	7
Función de POST al servidor	7
4.4 Partes importantes del servidor Node.js (server.js).....	8
Configuración de la base de datos	8
POST /api/puntuaciones - guardar o acumular puntaje	8
GET /api/puntuaciones/:rfid - consultar jugador	9

4.5 Partes importantes del launcher de Python	9
Esperar la tarjeta del ESP32 por Serial	9
Mandar puntaje al ESP32 para que lo suba al servidor	10
5. Configuración del Sistema	10
5.1 Tabla de configuración general	11
5.2 Esquema de la tabla SQL	11
6. Problemas Encontrados y Soluciones	12
6.1 Flujo de comunicación incorrecto	12
6.2 Falta de sqlcmd en la Mac	12
6.3 Contraseña incorrecta en server.js	12
6.4 Problemas con el router del salón	12
6.5 UUIDs subidos en formato incorrecto	12
6.6 Base de datos no creada en la Mac	13
6.7 PN532 no detectado por el ESP32	13
6.8 Firewall de Windows bloqueando el puerto 3000	13
7. Resultados Obtenidos	13
8. Conclusiones	14
9. Referencias	14

1. Introducción

Este proyecto integra varias tecnologías que se fueron viendo durante el semestre: el ESP32 como hardware central, tarjetas NFC/RFID para identificar jugadores, comunicación WiFi, un videojuego desarrollado en Python, un servidor Node.js como intermediario y una base de datos SQL Server corriendo en Docker. Todo esto junto simula un centro arcade real donde cada jugador tiene su puntaje guardado y acumulado en una base de datos compartida.

La idea es que cuando un jugador acerca su tarjeta al lector NFC, el sistema lo identifica, carga sus datos, le deja jugar y al terminar guarda su puntuación. Si la tarjeta no está registrada, el sistema la registra automáticamente con datos genéricos. Todo el procesamiento de la base de datos se hace en una Mac que funge como servidor central para todo el salón.

El flujo que se logró implementar fue: el juego en Python se comunica con el ESP32 por puerto Serial. El ESP32 lee la tarjeta NFC y manda el UID al juego. El juego consulta el servidor para saber si el jugador existe. Después de jugar, el juego manda el puntaje al ESP32 por Serial, y el ESP32 hace el POST al servidor para actualizar la base de datos. Esto es importante porque el flujo inicial estaba mal y tuvo que corregirse.

2. Marco Teórico e Investigación

2.1 NFC y RFID

RFID (Radio Frequency Identification) es una tecnología que permite identificar objetos o personas a través de ondas de radio sin necesidad de contacto físico. Funciona con dos componentes principales: una etiqueta (tag) que guarda información y un lector que la detecta y extrae esos datos.

NFC (Near Field Communication) es básicamente una evolución del RFID que opera a 13.56 MHz y requiere que los dispositivos estén muy cerca (generalmente menos de 4 cm). La diferencia principal es que NFC permite comunicación bidireccional, mientras que RFID clásico solo transmite en un sentido. Las tarjetas de transporte, llaves de hotel o las que usamos en este proyecto son ejemplos típicos de NFC.

En este proyecto se usó el módulo PN532, que es uno de los lectores NFC/RFID más comunes en proyectos con microcontroladores. Este módulo soporta los modos SPI, I2C y UART, y lo conectamos al ESP32 usando el protocolo I2C con los pines GPIO 21 (SDA) y GPIO 22 (SCL). Cada tarjeta tiene un UID (identificador único) que es el dato que usamos para identificar a los jugadores en la base de datos.

2.2 ESP32 y WiFi

El ESP32 es un microcontrolador de doble núcleo que integra WiFi 802.11 b/g/n y Bluetooth en un solo chip. Para este proyecto es perfecto porque necesitábamos que leyera las tarjetas NFC y al mismo tiempo se comunicara con el servidor por la red.

La librería WiFi.h que viene incluida con el soporte de ESP32 en Arduino IDE permite conectarse a redes existentes en modo STA (Station), que es como se llama cuando un dispositivo se conecta a un router como cliente normal. Una vez conectado, la librería HTTPClient.h permite hacer peticiones HTTP como GET y POST, que son exactamente lo que necesita el ESP32 para comunicarse con el servidor Node.js.

La comunicación Serial (puerto USB) se usó como canal de comunicación entre el ESP32 y el programa de Python que corre el juego. Por este canal el ESP32 manda el UID de la tarjeta leída, y Python le devuelve el puntaje a enviar al servidor cuando termina la partida. Este canal es importante porque permite que el juego y el ESP32 trabajen como un equipo sin que Python tenga que hacer las peticiones HTTP directamente.

2.3 Modulo PN532

El PN532 es un chip de NXP Semiconductors diseñado específicamente para aplicaciones NFC. Soporta el estándar ISO 14443 tipo A y B, que es el que usan la mayoría de las tarjetas de proximidad. En modo I2C, los jumpers SEL0 y SEL1 deben estar ambos en posición ON para que el módulo funcione correctamente en ese modo de comunicación.

La librería que se usa para comunicarse con este módulo desde Arduino es Adafruit_PN532, que simplifica bastante el proceso de lectura de tarjetas. La función readPassiveTargetID() es la que espera a que una tarjeta se acerque y devuelve su UID en formato hexadecimal.

2.4 Docker y SQL Server

Docker es una plataforma que permite ejecutar aplicaciones dentro de contenedores, que son como cajas aisladas que tienen todo lo necesario para que la aplicación funcione. La ventaja es

que no importa en que sistema operativo este el equipo: si tiene Docker, el contenedor va a funcionar igual.

Para este proyecto usamos la imagen `mcr.microsoft.com/azure-sql-edge` de Microsoft, que es una versión de SQL Server optimizada para dispositivos de borde y que funciona en Mac con chips Apple Silicon. El contenedor expone el puerto 1433 que es el estándar de SQL Server, y el servidor Node.js se conecta a él para hacer las consultas y actualizaciones.

2.5 Node.js como servidor intermediario

Node.js es un entorno de ejecución de JavaScript que permite crear servidores web de manera eficiente. En este proyecto el servidor Node.js actúa como puente entre el ESP32 y la base de datos: recibe las peticiones HTTP del microcontrolador, las procesa y ejecuta las operaciones necesarias en SQL Server.

Se uso el framework Express.js para definir las rutas de la API, el paquete `mssql` para conectarse a SQL Server, y `cors` para permitir que otros dispositivos en la red puedan hacer peticiones al servidor sin que el navegador las bloquee por políticas de seguridad.

2.6 Python y tkinter para el videojuego

El videojuego Snake se desarrolló en Python usando tkinter, que es la librería de interfaces graficas que viene incluida con Python. Se eligió Python porque es un lenguaje que permite desarrollar la lógica del juego y la interfaz gráfica de manera rápida, y además es multiplataforma (funciona igual en Windows y Mac).

La comunicación entre Python y el ESP32 se hace con la librería `pyserial`, que permite leer y escribir datos en el puerto Serial (USB). La librería `requests` se usa para hacer peticiones HTTP al servidor Node.js y consultar si un jugador ya existe en la base de datos antes de iniciar la partida.

3. Arquitectura del Sistema

La arquitectura final del sistema, después de las correcciones que se hicieron durante el desarrollo, queda de la siguiente manera:

Componente	Ubicación y función
ESP32 + PN532	Hardware físico. Lee tarjetas NFC y se comunica con Python por Serial y con el servidor por WiFi/HTTP
launcher_esp32_wifi.py	PC del equipo. Lanza el juego, consulta si el jugador existe y le pasa el puntaje al ESP32 para que lo envíe al servidor
snake_gui.py	PC del equipo. El videojuego Snake con interfaz grafica
server.js	Mac del equipo (servidor central). Recibe peticiones del ESP32, consulta y actualiza la base de datos
Docker + azure-sql-edge	Mac del equipo. Contenedor con la base de datos SQL Server compartida para todo el salón

El flujo de datos correcto es el siguiente:

- El ESP32 lee una tarjeta NFC y manda el UID por Serial a Python (formato: UID:XXXX)
- Python hace un GET al servidor para verificar si ese UID ya está en la base de datos
- Si no existe, se muestra una ventana de registro para que el jugador escriba su nombre
- El jugador juega Snake y cuando termina Python tiene el puntaje de esa sesión
- Python le manda el puntaje al ESP32 por Serial (formato: SCORE:RFID:puntos:nombre:usr)
- El ESP32 hace un POST al servidor con esos datos para actualizar el puntaje acumulado
- El servidor confirma con ACK:OK o ACK:ERROR de regreso al ESP32 por Serial

IMPORTANTE: El flujo inicial que se tenía estaba invertido. Antes el ESP32 mandaba los datos a Python y Python los mandaba al servidor, lo que causaba problemas de sincronización. El flujo correcto es que Python solo usa el Serial para pasarle el puntaje al ESP32, y el ESP32 es quien hace todas las peticiones HTTP al servidor.

4. Desarrollo del Proyecto

4.1 Configuración del servidor en la Mac

La Mac del equipo funciona como servidor central para todo el salón. Tiene Docker con el contenedor de azure-sql-edge y corre el server.js que atiende las peticiones de todos los ESP32 del salón.

Para crear el contenedor Docker en la Mac se usó:

```
docker run -e 'ACCEPT_EULA=Y' -e 'MSSQL_SA_PASSWORD=C0NTR453N1!4' \
-p 1433:1433 --name pruebaDb --hostname pruebaDB \
-d mcr.microsoft.com/azure-sql-edge
```

La base de datos y tabla se crearon con este SQL ejecutado desde el Exec de Docker Desktop o la terminal:

```
/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P 'C0NTR453N1!4'
1> CREATE DATABASE arcade_db;
2> go
1> USE arcade_db;
2> go
1> CREATE TABLE puntuaciones (
    ID INT PRIMARY KEY IDENTITY(1,1),
    NAME_DISP VARCHAR(50),
    USR VARCHAR(50),
    SCORE INT,
    LAST_GAME DATETIME DEFAULT GETDATE(),
    ID_RFID VARCHAR(50)
);
2> go
```

4.2 Configuración del Firewall en Mac

La Mac tenía el firewall activado, lo que bloqueaba las conexiones entrantes al puerto 3000 del servidor Node.js. Se tuvo que permitir que Node.js aceptara conexiones:

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --add $(which node)
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp $(which node)
```

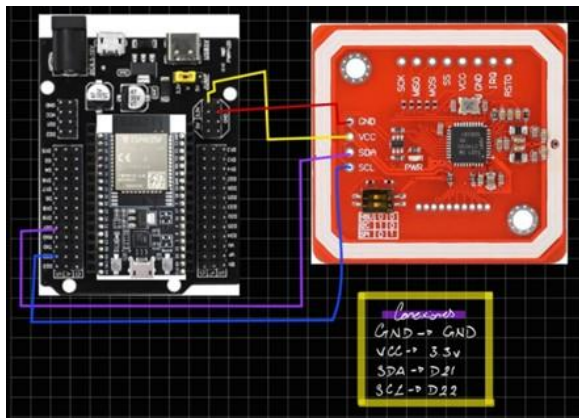
Para verificar que quedo bien configurado se usó:

```
sudo /usr/libexec/ApplicationFirewall/socketfilterfw --listapps
```

El resultado esperado es que aparezca node con el estado allow incoming connections.

4.3 Código del ESP32

El firmware del ESP32 se divide en dos partes principales: la lectura de tarjetas NFC y la comunicación con el servidor vía HTTP.



Configuración de pines y librerías

```
#include <Wire.h>
#include <Adafruit_PN532.h>
#include <WiFi.h>
#include <HttpClient.h>
#include <ArduinoJson.h>

// CONFIGURA ESTOS VALORES ANTES DE CORRER EL CODIGO PORQUE PUEDE SER OTRA RED O
OTRA IP

const char* WIFI_SSID      = "TP-Link_3262";          // Nombre de tu red WiFi
const char* WIFI_PASSWORD = "99428167";             // Contraseña WiFi

// IP de la Mac donde corre server.js (puerto 3000)
// Ejemplo: "192.168.0.105"
const char* SERVER_IP      = "192.168.0.104";
const int   SERVER_PORT    = 3000;
```

Lectura de tarjeta y envío de UID

Cuando se detecta una tarjeta, el ESP32 convierte el UID a hexadecimal y lo manda por Serial para que Python lo reciba:

```
// Convertir el UID a string hexadecimal
// El UID viene como arreglo de bytes, lo convertimos a texto
String rfid = "";
for (int i = 0; i < uidLen; i++) {
    if (uid[i] < 0x10) rfid += "0"; // padding para UIDs con digitos cortos
    rfid += String(uid[i], HEX);
}
rfid.toUpperCase();

// Mandamos el UID al programa Python por Serial
Serial.println("UID:" + rfid);
```

Recepción del puntaje y envío al servidor

Cuando Python termina la partida, manda el puntaje al ESP32 por Serial. El ESP32 lo parsea y hace el POST al servidor:

```
// Esperamos que Python nos mande algo con formato SCORE:rfid:puntos:name:usr
if (linea.startsWith("SCORE:")) {
    // Separamos cada campo usando ':' como delimitador
    // Esto nos da el rfid, los puntos, el nombre y el usuario
    String partes[5];
    int idx = 0;
    // ... parsing del mensaje ...
    bool ok = enviarScoreAlServidor(partes[1], partes[2].toInt(), partes[3],
partes[4]);
    // Le confirmamos a Python si salio bien o no
    Serial.println(ok ? "ACK:OK" : "ACK:ERROR");
}
```

Función de POST al servidor

```
bool enviarScoreAlServidor(String rfid, int score, String name, String usr) {
    HTTPClient http;
    // Apuntamos al endpoint de puntuaciones del servidor de la Mac
    http.begin(String(API_BASE) + "/api/puntuaciones");
    http.addHeader("Content-Type", "application/json");

    // Armamos el JSON con los datos del jugador
```

```

StaticJsonDocument<256> doc;
doc["id_rfid"]    = rfid;
doc["score"]     = score;
doc["name_disp"] = name;
doc["usr"]       = usr;

String body;
serializeJson(doc, body);
int code = http.POST(body);
http.end();
return (code == 200); // 200 = todo salio bien
}

```

4.4 Partes importantes del servidor Node.js (server.js)

El servidor corre en la Mac y atiende las peticiones de todos los equipos del salón. Las rutas más importantes son:

Configuración de la base de datos

```

// Aqui van las credenciales del Docker que esta corriendo en esta misma Mac
// server es 127.0.0.1 porque el Docker esta en la misma maquina
const dbConfig = {
  user:      'sa',
  password:  'C0NTR453N1!4', // misma que se puso al crear el contenedor
  server:    '127.0.0.1',    // localhost porque el Docker es local a la Mac
  database:  'arcade_db',
  port:      1433,
  options: { encrypt: false, trustServerCertificate: true }
};

```

POST /api/puntuaciones - guardar o acumular puntaje

```

app.post('/api/puntuaciones', async (req, res) => {
  const { name_disp, usr, score, id_rfid } = req.body;

  // Primero buscamos si ese RFID ya tiene registro en la BD
  let resultado = await pool.request()
    .input('id_rfid', sql.VarChar, id_rfid)
    .query('SELECT SCORE FROM puntuaciones WHERE ID_RFID = @id_rfid');

  if (resultado.recordset.length > 0) {
    // Ya existe: sumamos los puntos nuevos al total acumulado
    let nuevoTotal = resultado.recordset[0].SCORE + score;
    await pool.request()
      .input('nuevo_score', sql.Int, nuevoTotal)

```

```

        .input('id_rfid', sql.VarChar, id_rfid)
        .query('UPDATE puntuaciones SET SCORE=@nuevo_score WHERE
ID_RFID=@id_rfid');
    } else {
        // Es nuevo: creamos su primer registro con los puntos de esta partida
        await pool.request()
            .input('name_disp', sql.VarChar, name_disp)
            .input('score', sql.Int, score)
            .input('id_rfid', sql.VarChar, id_rfid)
            .query('INSERT INTO puntuaciones (NAME_DISP,USR,SCORE,ID_RFID) VALUES
(...)');
    }
    res.status(200).json({ status: 'success', total_score: nuevoTotal });
});

```

GET /api/puntuaciones/:rfid - consultar jugador

```

app.get('/api/puntuaciones/:rfid', async (req, res) => {
    // Buscamos el jugador por su RFID en la base de datos
    let resultado = await pool.request()
        .input('id_rfid', sql.VarChar, req.params.rfid)
        .query('SELECT SCORE, USR FROM puntuaciones WHERE ID_RFID = @id_rfid');

    if (resultado.recordset.length > 0) {
        // Lo encontramos: mandamos sus datos para que el juego los muestre
        res.json({ existe: true, score: resultado.recordset[0].SCORE, ... });
    } else {
        // No existe: el launcher lo va a registrar
        res.json({ existe: false, score: 0 });
    }
});

```

4.5 Partes importantes del launcher de Python

El launcher.py es el programa que coordina todo del lado de la PC: espera el UID del ESP32, consulta el servidor, lanza el juego y le manda el puntaje al ESP32 al terminar.

Esperar la tarjeta del ESP32 por Serial

```

def esperar_tarjeta(ser):
    # Nos quedamos leyendo el puerto Serial hasta que llegue un UID
    # El ESP32 manda 'UID:XXXXXXXXXX' cuando detecta una tarjeta
    while True:
        linea = ser.readline().decode('utf-8', errors='ignore').strip()
        if linea.startswith('UID:'):

```

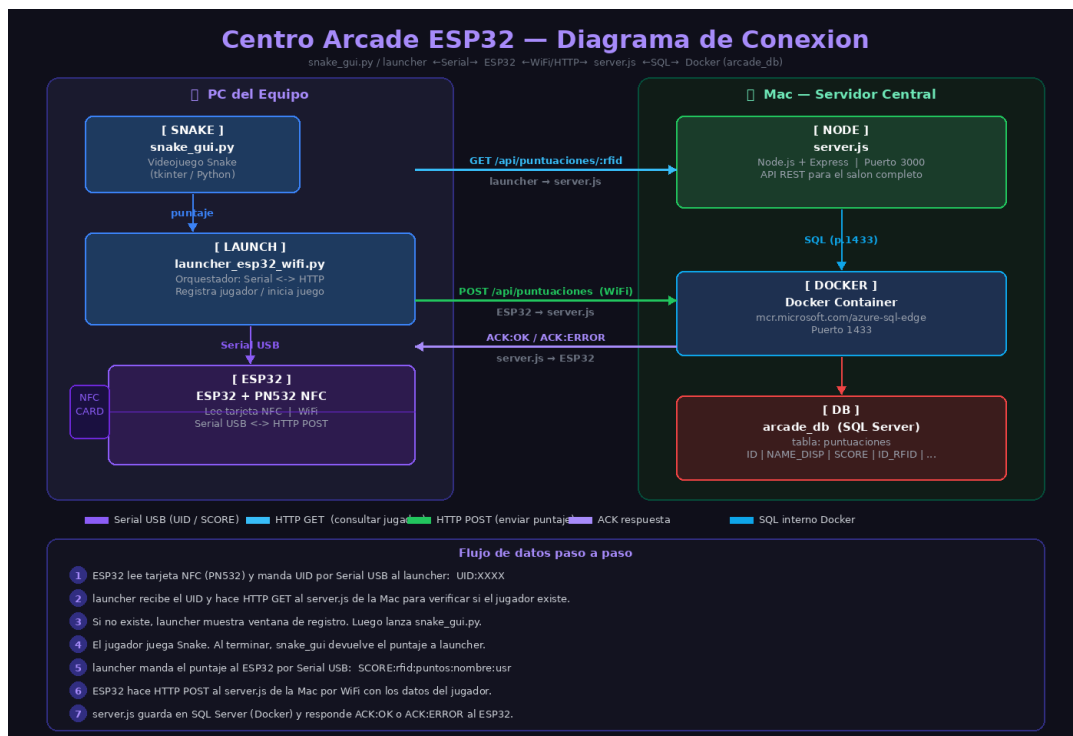
```
uid = linea[4:] # quitamos el prefijo 'UID:'
return uid      # regresamos solo el hexadecimal
```

Mandar puntaje al ESP32 para que lo suba al servidor

```
def enviar_score_via_esp32(ser, jugador, puntos):
    # Armamos el mensaje con todos los datos separados por ':'
    # El ESP32 va a parsear esto y hacer el POST al servidor
    mensaje =
f"SCORE:{jugador['id_rfid']}:{puntos}:{jugador['name_disp']}:{jugador['usr']}\n"
    ser.write(mensaje.encode('utf-8'))

# Esperamos que el ESP32 nos confirme si logro subir los datos
deadline = time.time() + ACK_TIMEOUT
while time.time() < deadline:
    linea = ser.readline().decode('utf-8').strip()
    if linea == 'ACK:OK':
        return True # el ESP32 confirmo que el servidor recibio los datos
    elif linea == 'ACK:ERROR':
        return False # algo salio mal en el camino
```

5. Configuración del Sistema



5.1 Tabla de configuración general

Servidor central	Mac del equipo
IP del servidor	192.168.0.104 (puede variar según la red ip donde este el servidor, ósea donde la computadora tenga el servidor arrancando)
Puerto servidor Node.js	3000
Puerto SQL Server Docker	1433
Base de datos	arcade_db
Tabla SQL	puntuaciones
Imagen Docker	mcr.microsoft.com/azure-sql-edge
Usuario BD	sa
Puerto Serial ESP32	COM10 (Windows) / variable según equipo
Velocidad Serial	115200 baudios
Pines NFC (I2C)	SDA: GPIO 21, SCL: GPIO 22
Modo NFC	I2C (ambos jumpers en ON)

5.2 Esquema de la tabla SQL

Campo	Tipo	Restricción	Descripción		
ID	INT	PK, IDENTITY	Clave autoincremental		
NAME_DISP	VARCHAR(50)		Nombre del dispositivo		
USR	VARCHAR(50)		Nombre elegido por el jugador		
SCORE	INT		Puntaje acumulado total		
LAST_GAME	DATETIME	DEFAULT GETDATE()	Fecha y hora de la ultima partida		
ID_RFID	VARCHAR(50)		UID hexadecimal de la tarjeta		

6. Problemas Encontrados y Soluciones

6.1 Flujo de comunicación incorrecto

El error más importante del proyecto fue tener el flujo al revés. Originalmente Python mandaba los datos al servidor directamente, y el ESP32 solo leía la tarjeta. El problema era que el ESP32 no estaba involucrado en el ciclo completo y el servidor no recibe las peticiones de forma consistente.

La solución fue reordenar el flujo: Python le pasa el puntaje al ESP32 por Serial, y el ESP32 es quien hace el POST al servidor. Esto funciona mejor porque el ESP32 ya tiene la conexión WiFi establecida y puede manejar las peticiones HTTP de manera independiente.

6.2 Falta de sqlcmd en la Mac

Al intentar conectarse a la base de datos desde la terminal de la Mac con la ruta `/opt/mssql-tools/bin/sqlcmd`, el sistema decía que no encontraba el archivo. Esto se debe a que la imagen `azure-sql-edge` en Mac con chip Apple Silicon tenía la herramienta en una ruta diferente o simplemente no estaba incluida.

La solución fue usar el comando `find` dentro del contenedor para localizar `sqlcmd`:

```
docker exec -it pruebaDb find / -name 'sqlcmd' 2>/dev/null
```

La alternativa más práctica fue instalar Azure Data Studio en la Mac, que tiene interfaz gráfica y no necesita `sqlcmd` para ejecutar consultas SQL.

6.3 Contraseña incorrecta en server.js

El servidor Node.js en la Mac tenía una contraseña diferente en el archivo de configuración de la base de datos. El campo `password` en `dbConfig` no coincidía con la contraseña que se usó al crear el contenedor Docker, lo que causaba un error de autenticación al intentar conectarse.

¡La solución fue editar `server.js` y asegurarse de que la contraseña fuera exactamente `CONTR453N1!4`, que es la que se especificó en la variable `MSSQL_SA_PASSWORD` al crear el contenedor.

6.4 Problemas con el router del salón

Se presentaron varios problemas relacionados con el router al que todos los equipos del salón se conectaron:

- La contraseña del router fue cambiada en algún momento, lo que desconectó a varios equipos y obligó a reconectarse con las nuevas credenciales.
- Al desconectarse y reconectarse, algunas computadoras recibieron una IP diferente del DHCP, lo que hizo que la IP del servidor de la Mac ya no fuera válida y hubiera que actualizar `server.js` y los archivos del ESP32 con la nueva IP.
- El router tenía desactivado el DHCP en algún momento, lo que impide que los dispositivos obtuvieran IP automáticamente y no podían conectarse a la red. Se tuvo que reiniciar el router a valores de fábrica y activar el DHCP nuevamente.

La lección de este problema es que en proyectos donde varios equipos dependen de una IP fija, lo ideal es asignar una IP estática al servidor o configurar una reserva DHCP para que siempre obtenga la misma IP.

6.5 UUIDs subidos en formato incorrecto

Algunos equipos del salón tenían problemas donde el UID de la tarjeta NFC se sube a la base de datos en un formato diferente al esperado. El UID que lee el ESP32 es un arreglo de bytes que hay que convertir a hexadecimal como texto, y si esta conversión no se hace igual en todos los equipos, el mismo RFID puede quedar guardado de formas distintas.

El equipo propio no tuvo este problema porque la conversión se hacía correctamente con padding de ceros cuando era necesario (por ejemplo, si un byte vale 9, se guardaba como 09 y no como 9 para mantener consistencia). Los equipos con el problema tuvieron que limpiar los registros incorrectos de la base de datos y asegurarse de hacer la conversión de la misma forma.

6.6 Base de datos no creada en la Mac

En un punto del proyecto la Mac no tenía creada la base de datos `arcade_db` ni la tabla puntuaciones, lo que causaba que el servidor respondiera con error 500 a todas las peticiones. El servidor Node.js podía conectarse al contenedor Docker, pero la base de datos especificada no existe.

Se resolvió ejecutando el SQL de creación directamente desde el contenedor, ya sea por el Exec de Docker Desktop o desde la terminal de Mac con `docker Exec`.

6.7 PN532 no detectado por el ESP32

Al iniciar el ESP32 aparecía el mensaje `ERROR: PN532 no detectado`, aunque los cables se veían bien conectados físicamente. Las causas posibles son los jumpers del módulo en posición incorrecta o cables mal insertados.

La solución fue verificar que ambos jumpers del PN532 (SEL0 y SEL1) estuvieran en posición ON para modo I2C, y subir un sketch de escáner I2C para verificar que el ESP32 detectara el dispositivo en la dirección 0x24 antes de continuar con el Código completo.

6.8 Firewall de Windows bloqueando el puerto 3000

Cuando el equipo se conectó a un router diferente al de casa, Windows lo clasificó como red pública y el firewall bloqueó las conexiones entrantes al puerto 3000. El ESP32 recibía error `HTTP -1` al intentar conectarse.

La solución fue agregar una regla al firewall de Windows para permitir el puerto 3000:

```
New-NetFirewallRule -DisplayName 'Arcade API 3000' -Direction Inbound -Protocol TCP -LocalPort 3000 -Action Allow
```

7. Resultados Obtenidos

Al terminar el proyecto y hacer las pruebas finales con todos los equipos del salón, se lograron los siguientes resultados:

- El ESP32 lee correctamente las tarjetas NFC y manda el UID por Serial a Python en formato hexadecimal estandarizado.
- El launcher de Python recibe el UID, consulta el servidor y muestra la ventana de registro si la tarjeta no está en la base de datos.
- El juego Snake funciona correctamente y al terminar devuelve el puntaje al launcher para que se lo mande al ESP32.
- El ESP32 recibe el puntaje por Serial y hace el POST al servidor de la Mac, que actualiza el puntaje acumulado en la base de datos.

- El servidor de la Mac responde correctamente a las peticiones de todos los equipos del salón que se conectaron a la misma red.
- Se validó el funcionamiento cruzado: el RFID de un equipo funciona en el juego de otro equipo del salón correctamente.

8. Conclusiones

Eduardo: Este proyecto me mostro lo complicado que puede ser integrar varios sistemas al mismo tiempo. El mayor aprendizaje fue que el orden del flujo de datos importa mucho: tener el flujo al revés causaba problemas que no eran obvios al principio. Una vez que se corrigió la arquitectura, todo empezó a funcionar mucho mejor.

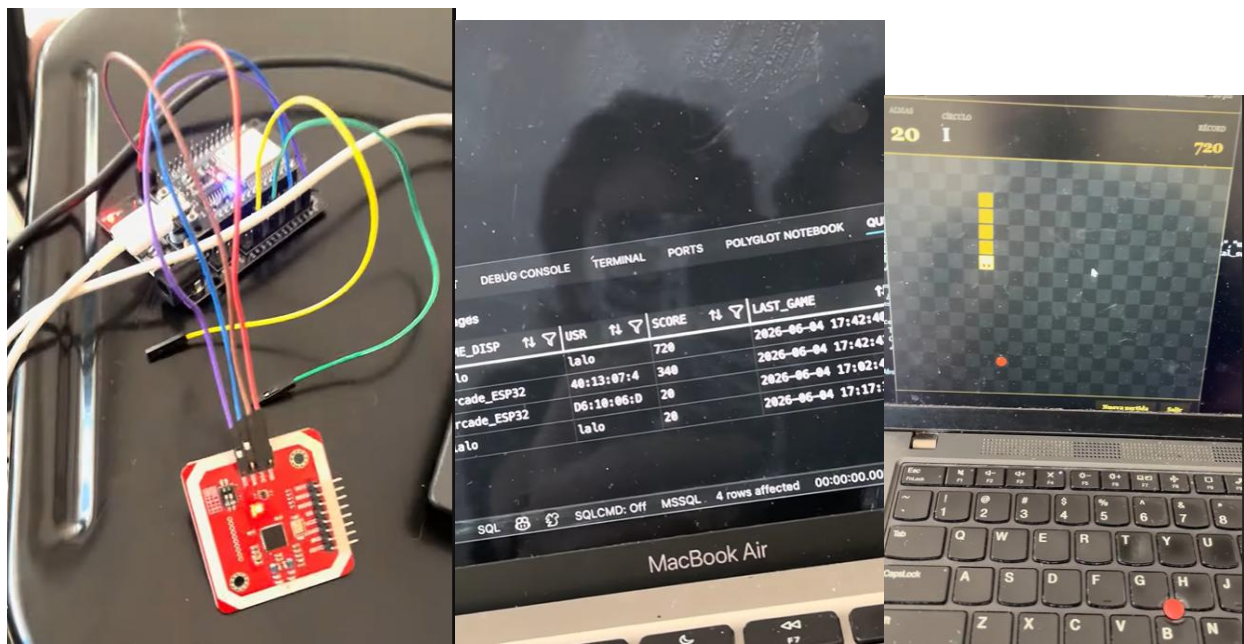
Damian: Los problemas con el router me hicieron entender por qué en entornos reales siempre se usan IPs estáticas para los servidores. Depender de que el DHCP siempre asigne la misma IP es un error de diseño que en este proyecto nos costó tiempo. Docker fue muy útil para que todos los equipos usaran exactamente la misma base de datos sin tener que instalar SQL Server en cada computadora.

Itzel: La parte de NFC fue interesante porque parece sencilla, pero tiene detalles que pueden arruinar todo, como el modo de los jumpers o el formato en que se guarda el UID. El proyecto final integra bien todo lo que se vio en el semestre: redes inalámbricas, bases de datos, hardware y desarrollo de software.

9. Referencias, Videos y GitHub

YouTube Video Funcionamiento: <https://youtube.com/shorts/C8QyNg6J734?feature=share>

Github: https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/Esp32DockerArcaDePy



Adrianus Share...

Sesión terminada. Puntos ganados: 30

Enviando score al ESP32 para que lo mande al servidor...

→ Enviando al ESP32: SCORE:491C3307:30:lalo:lalo

Esperando confirmación del ESP32 (máx 10s)...

[ESP32] [Serial] Recibido: SCORE:491C3307:30:lalo:lalo

[ESP32] [Score] RFID=491C3307 | Pts=30 | Nombre=lalo

[ESP32] [HTTP] POST http://192.168.0.103:3000/api/puntuaciones

[ESP32] [HTTP] Body: {"id_rfid":"491C3307","score":30,"name_disp":"lalo"}

[ESP32] [HTTP] Respuesta 200: {"status":"success","action":"update"}

[ESP32] [Score] ✓ Score enviado al servidor correctamente.

[ESP32] ACK:OK

✓ ESP32 confirmó: score enviado al servidor correctamente.